Logika Układów Cyfrowych

Dawid Kocyła i Ida Langowska

20 Październik 2025

Spis treści

1 Cel ćwiczenia **2**

2 Zadanie 1 – Komparator 2-bitowy **2**

3 Zadanie 2 – Układ realizujący sumę lub iloczyn logiczny **5**

4 Zadanie 3 – Pełny sumator 1-bitowy **6**

5 Zadanie 4 – Pełny subtraktor 1-bitowy **7**

6 Zadanie 5 – 4-bitowy kaskadowy sumator równoległy **8**

7 Zadanie 6 – Uzupełnienie do dwóch z wykorzystaniem 7483 **11**

8 Zadanie 7 – Sterowane przepuszczenie lub negacja bitu **13**

9 Zadanie 8 – Detekcja liczby z przedziału 5–10 **15**

10 Wnioski końcowe **17**

1 Cel ćwiczenia

Celem ćwiczenia było projektowanie i minimalizacja układów kombinacyjnych: komparatora, układu AND/OR, sumatorów, subtraktora, układów uzupełnienia do dwóch oraz detektora przedziału 5–10.

2 Zadanie 1 – Komparator 2-bitowy

2.1 Treść zadania

Zaprojektować komparator dwóch liczb 2-bitowych $A = A_1A_0$ oraz $B = B_1B_0$ w dwóch wariantach:

a) dwa wyjścia: 00 $A = B$, 01 $A > B$, 10 $A < B$, 11 stan nieokreślony,

b) trzy wyjścia, z których każde odpowiada osobno relacji $A = B$, $A > B$ albo $A < B$.

2.2 Tabela prawdy

A_1	A_0	B_1	B_0	Y_1	Y_0	E	G	L
0	0	0	0	0	0	1	0	0
0	0	0	1	1	0	0	0	1
0	0	1	0	1	0	0	0	1
0	0	1	1	1	0	0	0	1
0	1	0	0	0	1	0	1	0
0	1	0	1	0	0	1	0	0
0	1	1	0	1	0	0	0	1
0	1	1	1	1	0	0	0	1
1	0	0	0	0	1	0	1	0
1	0	0	1	0	1	0	1	0
1	0	1	0	0	0	1	0	0
1	0	1	1	1	0	0	0	1
1	1	0	0	0	1	0	1	0
1	1	0	1	0	1	0	1	0
1	1	1	0	0	1	0	1	0
1	1	1	1	0	0	1	0	0

W wariancie a) zachodzi $Y_0 = G$ oraz $Y_1 = L$.

2.3 Minimalizacja

$$\begin{aligned} E &= (\overline{A_1B_1} + A_1B_1)(\overline{A_0B_0} + A_0B_0) \\ &= (A_1 \odot B_1)(A_0 \odot B_0). \end{aligned}$$

$$\begin{aligned} G &= A_1\overline{B_1} + (A_1B_1 + \overline{A_1B_1})A_0\overline{B_0} \\ &= A_1(\overline{B_1} + A_0\overline{B_0}) + \overline{B_1}A_0\overline{B_0} \\ &= A_1\overline{B_1} + A_0A_1\overline{B_0} + A_0\overline{B_1B_0}. \end{aligned}$$

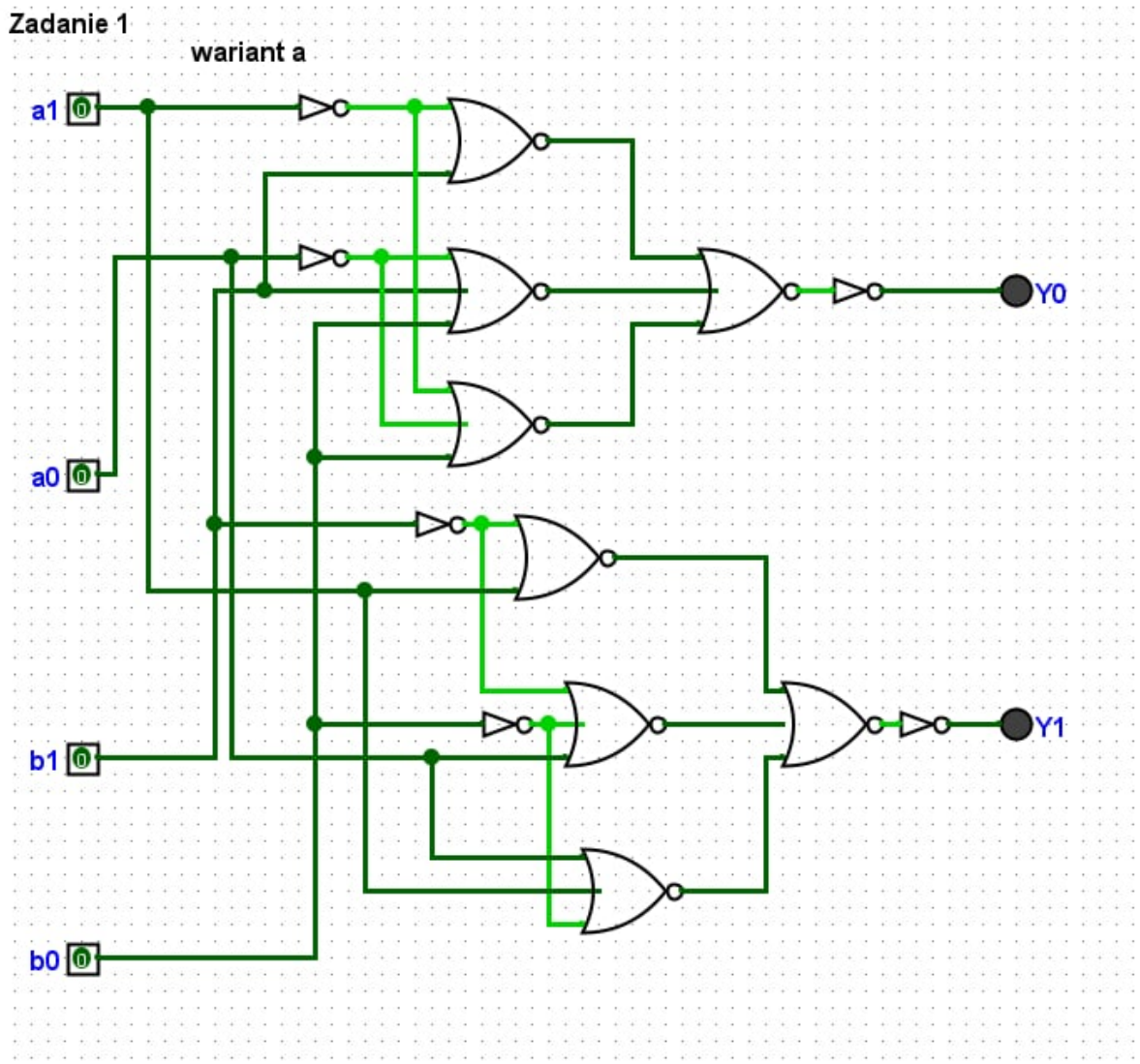
$$\begin{aligned} L &= \overline{A_1}B_1 + (A_1B_1 + \overline{A_1B_1})\overline{A_0}B_0 \\ &= B_1(\overline{A_1} + \overline{A_0}B_0) + \overline{A_1A_0}B_0 \\ &= \overline{A_1}B_1 + \overline{A_0}B_0B_1 + \overline{A_1A_0}B_0. \end{aligned}$$

Zastosowano $x + \overline{x}y = x + y$. Dla wariantu a):

$$Y_0 = G, \qquad Y_1 = L.$$

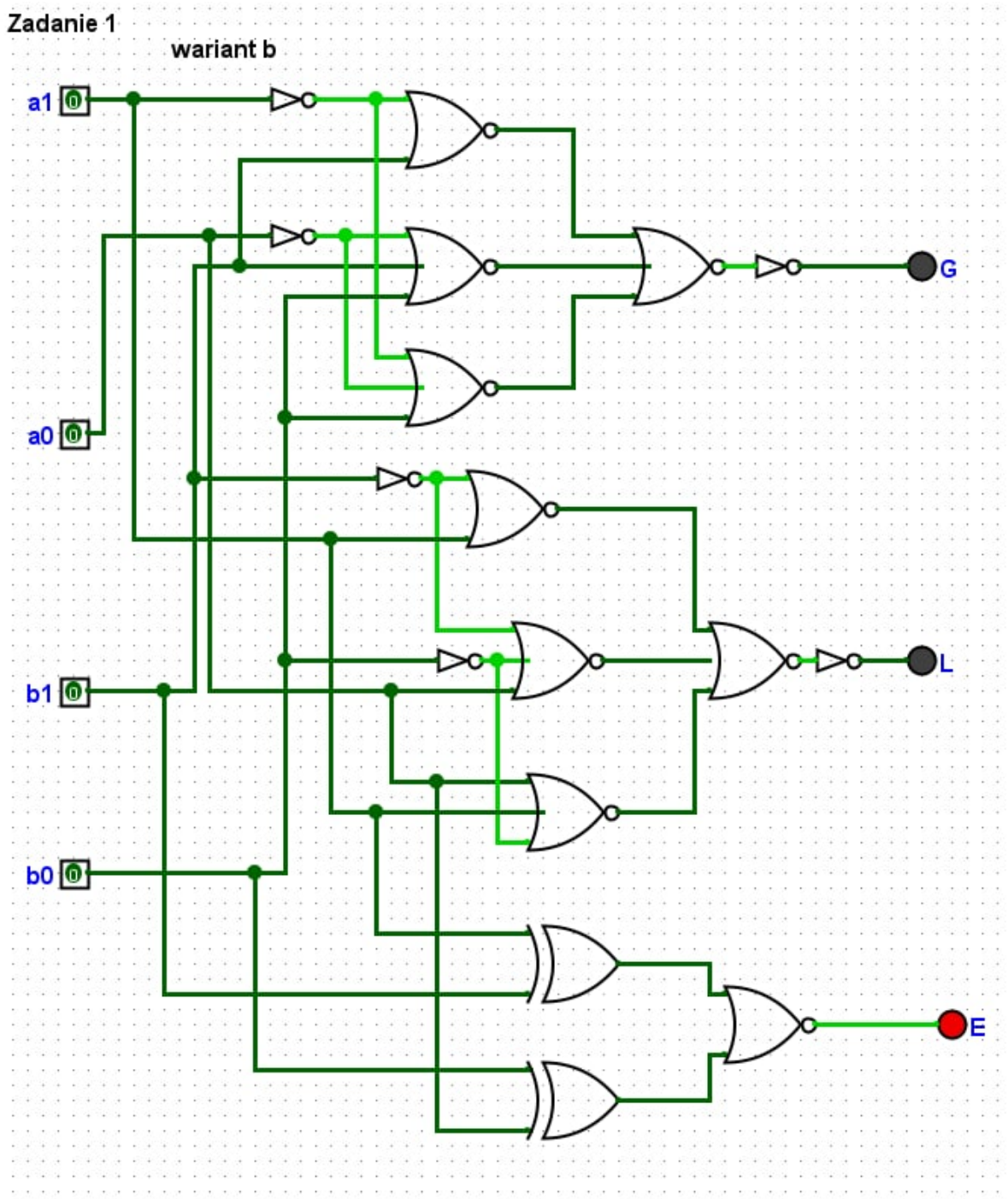
2.4 Realizacja – wariant a)

Na rysunku przedstawiono realizację z dwoma wyjściami kodującymi wynik porównania.



2.5 Realizacja – wariant b)

W wariancie b) zastosowano trzy niezależne wyjścia: G, L i E.



2.6 Weryfikacja

Dla $(A, B) = (00, 00)$ aktywne jest E, dla $(11, 10)$ – G, a dla $(01, 10)$ – L. W wariancie a) kod 11 nie występuje.

2.7 Wnioski

Trzy wyjścia wskazują relacje bezpośrednio; dwa pozwalają je zakodować mniejszą liczbą linii.

3 Zadanie 2 – Układ realizujący sumę lub iloczyn logiczny

3.1 Treść zadania

Zaprojektować układ, który w zależności od sygnału sterującego realizuje sumę logiczną lub iloczyn logiczny.

3.2 Założenia i tabela prawdy

Przyjęto:

$$S = 0 \Rightarrow Y = A \cdot B, \qquad S = 1 \Rightarrow Y = A + B.$$

S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

3.3 Minimalizacja

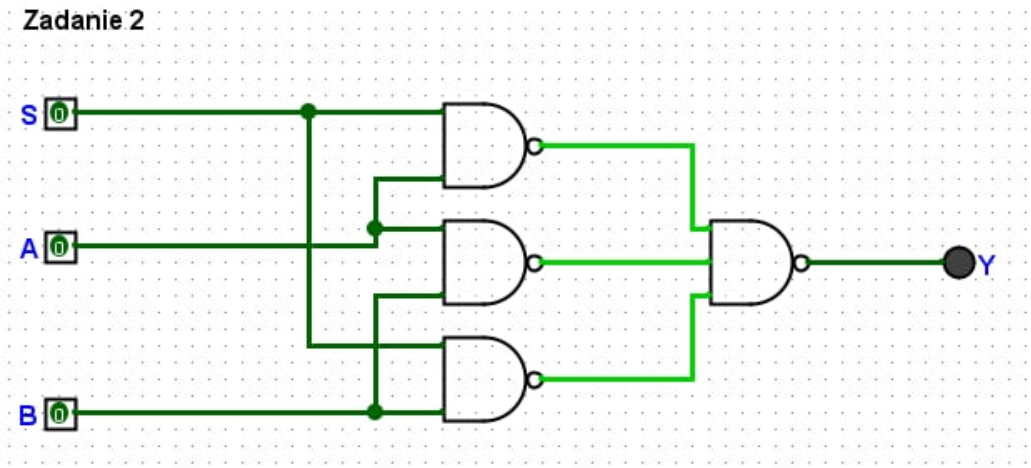
$$\begin{aligned} Y &= \overline{S}AB + S(A+B) \\ &= \overline{S}AB + SA + SB \\ &= AB + SA + SB = AB + S(A+B). \end{aligned}$$

Zastosowano $\overline{S}AB + SA = AB + SA$. Postać NAND–NAND:

$$Y = \overline{\overline{AB} \cdot \overline{AS} \cdot \overline{BS}}.$$

3.4 Realizacja

Schemat wykorzystuje bramki NAND. Trzy pierwsze bramki tworzą zanegowane iloczyny AS, AB i BS, a ostatnia bramka NAND realizuje ich sumę zgodnie z prawem de Morgana.



3.5 Wnioski

Sygnał S wybiera AND lub OR; obie funkcje zrealizowano bramkami NAND.

4 Zadanie 3 – Pełny sumator 1-bitowy

4.1 Treść zadania

Zaprojektować pełny sumator 1-bitowy.

4.2 Tabela prawdy

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

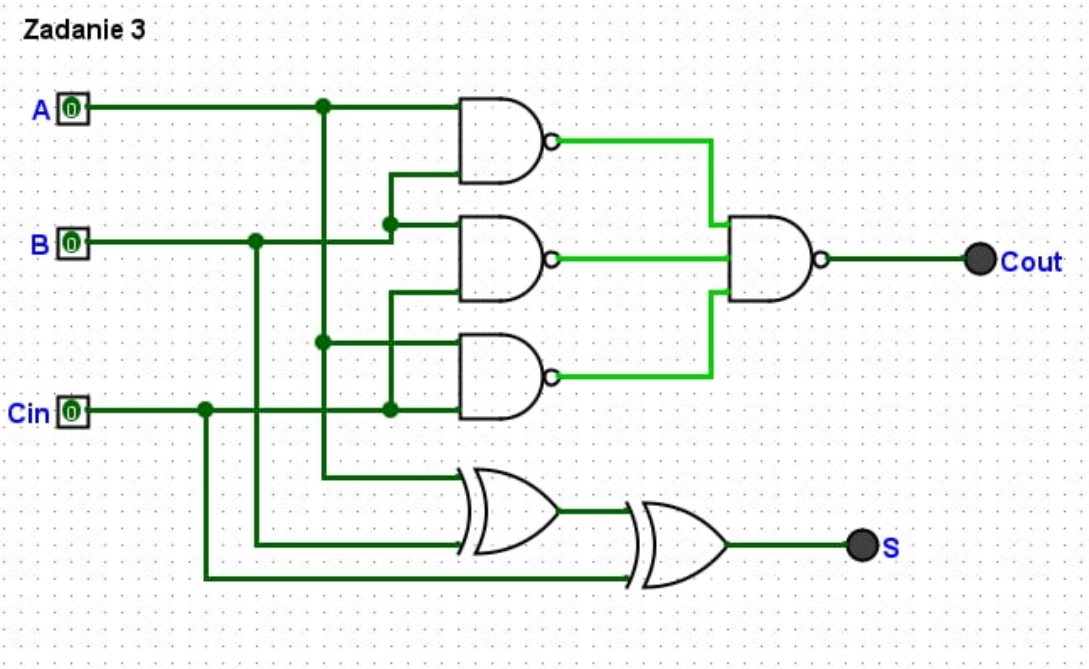
4.3 Minimalizacja

$$\begin{aligned} S &= \overline{A}\overline{B}C_{\text{in}} + \overline{A}B\overline{C_{\text{in}}} + A\overline{B}\overline{C_{\text{in}}} + ABC_{\text{in}} \\ &= C_{\text{in}}(\overline{A}\overline{B} + AB) + \overline{C_{\text{in}}}(\overline{A}B + A\overline{B}) \\ &= (A \oplus B) \oplus C_{\text{in}}. \end{aligned}$$

$$\begin{aligned} C_{\text{out}} &= AB\overline{C_{\text{in}}} + ABC_{\text{in}} + A\overline{B}C_{\text{in}} + \overline{A}BC_{\text{in}} \\ &= AB + C_{\text{in}}(A\overline{B} + \overline{A}B) \\ &= AB + C_{\text{in}}(A \oplus B) = AB + AC_{\text{in}} + BC_{\text{in}}. \end{aligned}$$

4.4 Realizacja

Dolna gałąź z dwiema bramkami XOR tworzy bit sumy. Górna część układu realizuje trzy iloczyny odpowiedzialne za przeniesienie, zapisane w strukturze NAND–NAND.



4.5 Wnioski

Pełny sumator uwzględnia trzy bity wejściowe. Przeniesienie pojawia się wtedy, gdy co najmniej dwa z trzech wejść mają wartość 1.

5 Zadanie 4 – Pełny subtraktor 1-bitowy

5.1 Treść zadania

Zaprojektować pełny subtraktor 1-bitowy.

5.2 Tabela prawdy

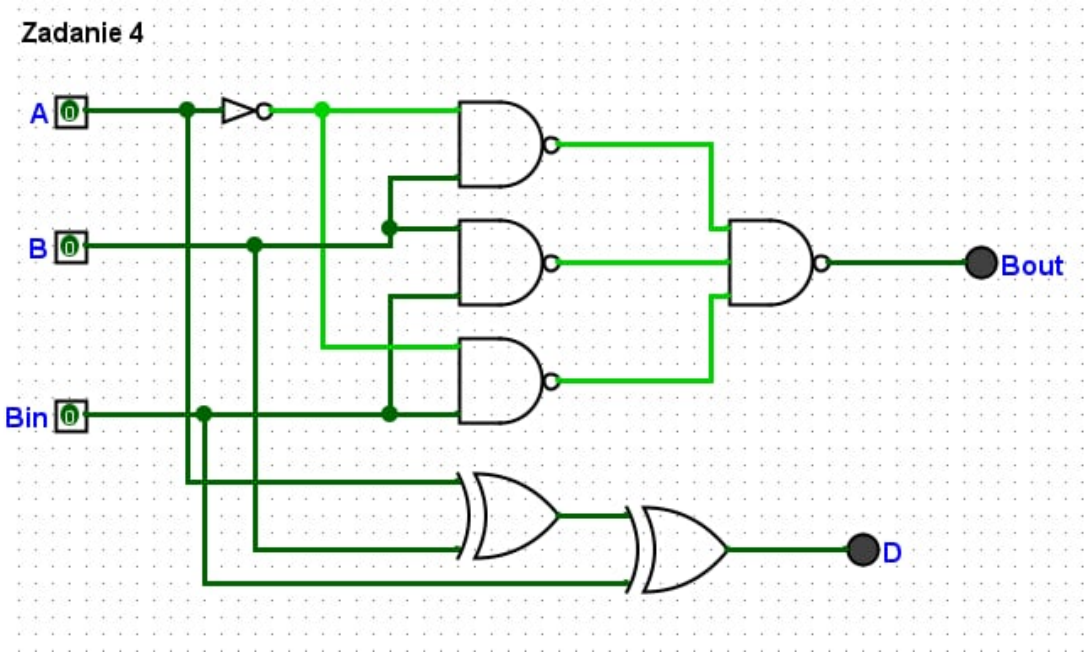
A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

5.3 Minimalizacja

$$\begin{aligned} D &= \overline{A}\overline{B}B_{\text{in}} + \overline{A}B\overline{B_{\text{in}}} + A\overline{B}\,\overline{B_{\text{in}}} + ABB_{\text{in}} \\ &= B_{\text{in}}(\overline{A}\overline{B} + AB) + \overline{B_{\text{in}}}(\overline{A}B + A\overline{B}) \\ &= (A \oplus B) \oplus B_{\text{in}}. \\ B_{\text{out}} &= \overline{A}\overline{B}B_{\text{in}} + \overline{A}B\overline{B_{\text{in}}} + \overline{A}BB_{\text{in}} + ABB_{\text{in}} \\ &= \overline{A}B + \overline{A}\overline{B}B_{\text{in}} + ABB_{\text{in}} \\ &= \overline{A}B + \overline{A}B_{\text{in}} + BB_{\text{in}}. \end{aligned}$$

5.4 Realizacja

Bit różnicy jest generowany przez dwie bramki XOR. W torze pożyczki wykorzystano negację A i strukturę NAND–NAND odpowiadającą powyższej funkcji.



5.5 Wnioski

Różnicę wyznaczają bramki XOR, a pożyczkę – funkcja uwzględniająca zanegowaną odjemną A.

6 Zadanie 5 – 4-bitowy kaskadowy sumator równoległy

6.1 Treść zadania

Z gotowych modułów zrealizować 4-bitowy kaskadowy sumator równoległy.

6.2 Zasada działania

Cztery pełne sumatory 1-bitowe połączono kaskadowo. Przeniesienie z pozycji mniej znaczącej jest wejściem przeniesienia kolejnego stopnia:

$$(A_0, B_0, C_0) \to (S_0, C_1),$$

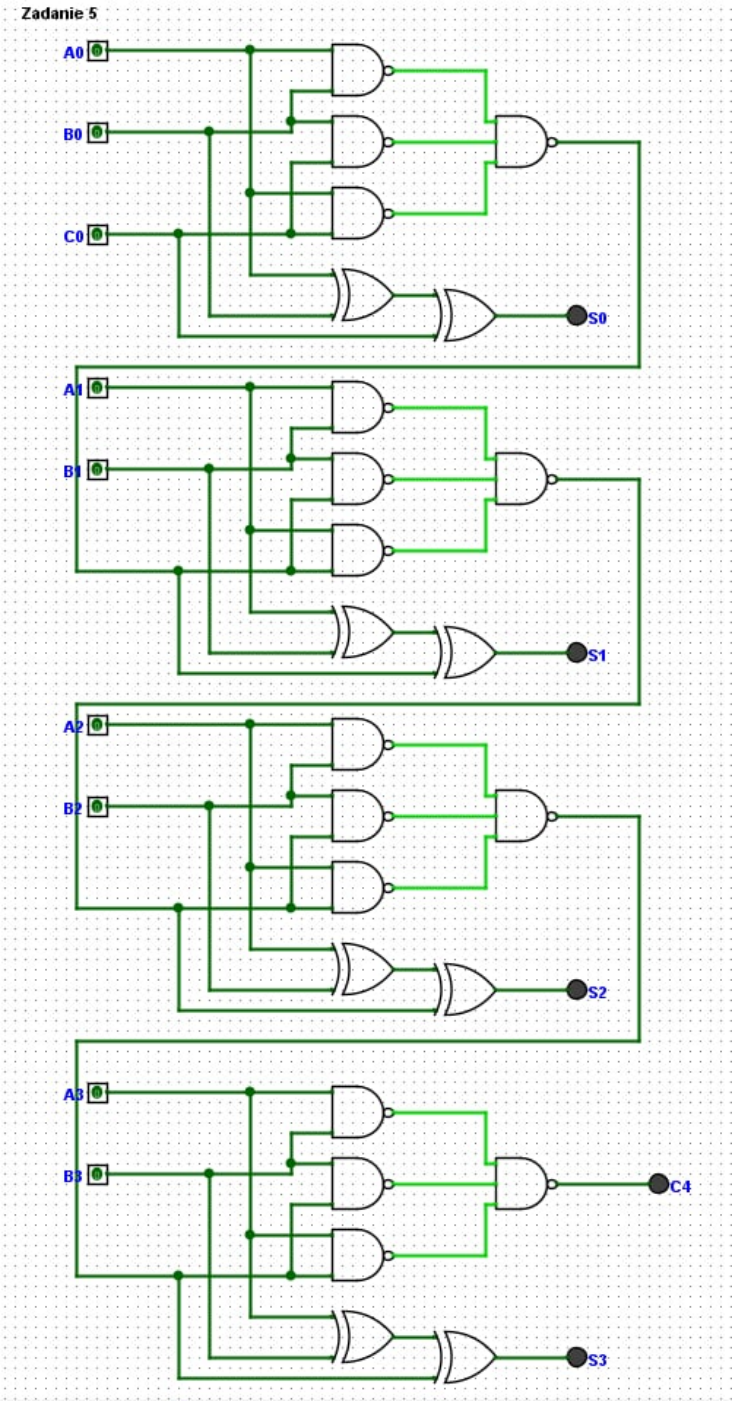
$$(A_1, B_1, C_1) \to (S_1, C_2),$$
$$(A_2, B_2, C_2) \to (S_2, C_3),$$
$$(A_3, B_3, C_3) \to (S_3, C_4).$$

6.3 Uproszczenie zależności

$$P_i = A_i \oplus B_i, \qquad G_i = A_i B_i,$$
$$S_i = P_i \oplus C_i, \qquad C_{i+1} = G_i + P_i C_i, \quad i = 0, 1, 2, 3.$$

6.4 Realizacja

Na schemacie widać cztery identyczne stopnie pełnego sumatora oraz propagację przeniesienia od C_0 do C_4.



6.5 Przykładowa weryfikacja

A	B	C_0	$C_4S_3S_2S_1S_0$
0000	0000	0	00000
0011	0101	0	01000
1111	0001	0	10000
1010	0101	0	01111
0111	1000	1	10000

6.6 Wnioski

Układ dodaje dwie liczby 4-bitowe i C_0. Propagacja przeniesienia przez kolejne stopnie zwiększa opóźnienie wyniku.

7 Zadanie 6 – Uzupełnienie do dwóch z wykorzystaniem 7483

7.1 Treść zadania

Za pomocą układu 7483 zrealizować układ generujący uzupełnienie do dwóch liczby 4-bitowej.

7.2 Zasada działania

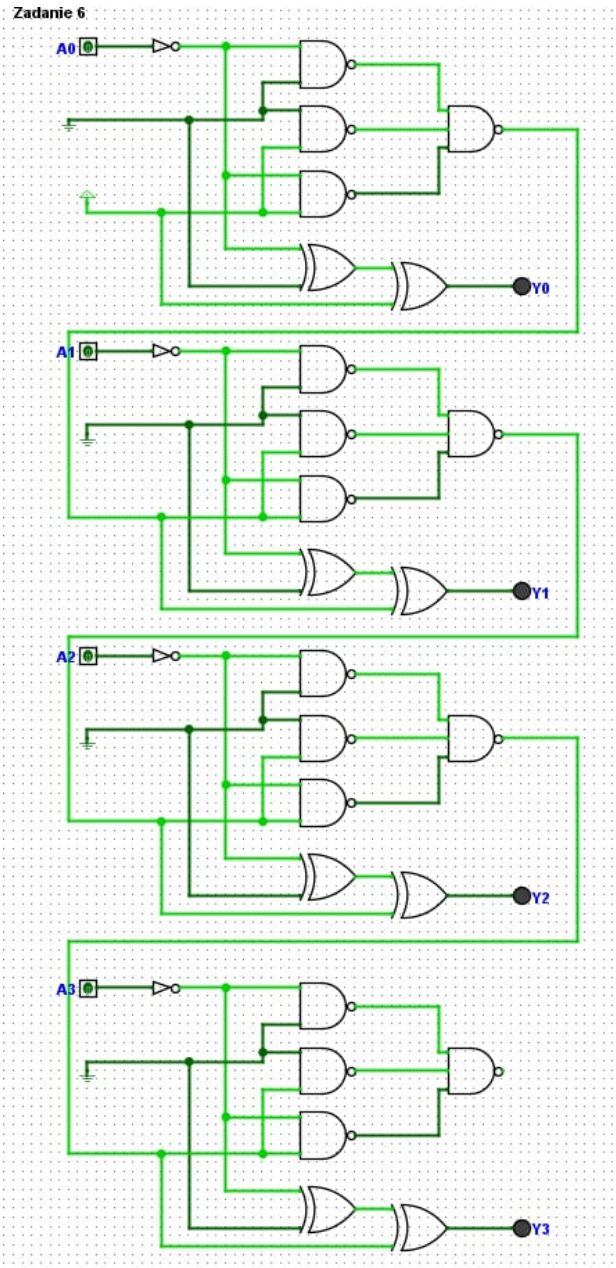
Dla liczby 4-bitowej $A = A_3A_2A_1A_0$ uzupełnienie do dwóch wyznacza się z zależności

$$Y = (\overline{A} + 1) \bmod 16 = (15 - A + 1) \bmod 16 = (-A) \bmod 16.$$

Do sumatora podaje się zanegowane bity A, $B = 0000$ oraz $C_0 = 1$. Przeniesienie końcowe pomija się, zachowując wynik modulo 16.

7.3 Realizacja

Schemat przedstawia negację bitów wejściowych i dodanie jedynki za pomocą czterech kaskadowo połączonych sumatorów 1-bitowych.



7.4 Tabela testów

A	$Y = \overline{A} + 1$
0000	0000
0001	1111
0010	1110
0101	1011
1000	1000
1111	0001

7.5 Wnioski

Zanegowanie bitów i dodanie jedynki daje 4-bitowe uzupełnienie do dwóch.

8 Zadanie 7 – Sterowane przepuszczenie lub negacja bitu

8.1 Treść zadania

Zrealizować układ, który w zależności od sygnału sterującego daje na wyjściu sygnał wejściowy albo jego negację. Korzystając z takiego układu zaprojektować układ realizujący uzupełnienie do dwóch liczby 4-bitowej.

8.2 Część A – układ 1-bitowy

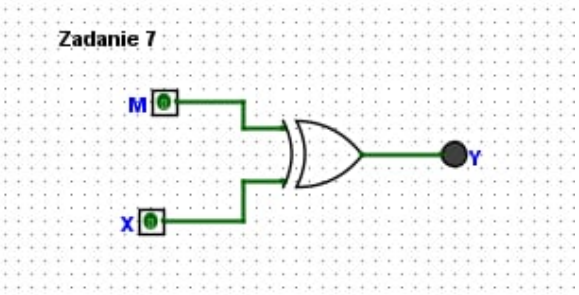
Przyjęto, że $M = 0$ oznacza przepuszczenie sygnału, a $M = 1$ jego negację.

M	X	Y
0	0	0
0	1	1
1	0	1
1	1	0

Minimalizacja:

$$Y = \overline{M}X + M\overline{X} = X \oplus M.$$

Zatem pojedyncza bramka XOR realizuje dokładnie wymaganą funkcję.



8.3 Część B – rozszerzenie do liczby 4-bitowej

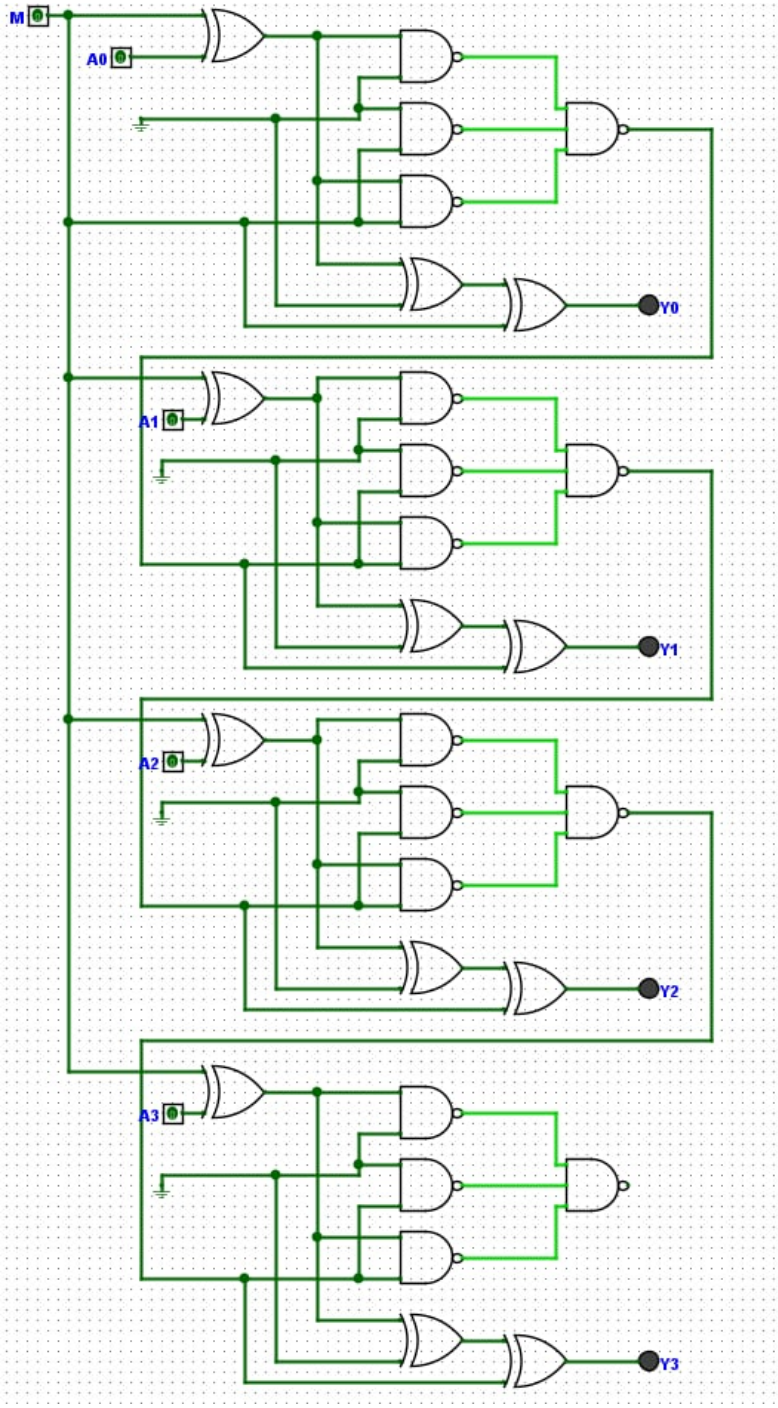
Cztery identyczne układy XOR można zastosować równolegle do wszystkich bitów. Aby otrzymać sterowaną negację arytmetyczną, do wyniku należy dodać sygnał M:

$$Y = (A \oplus M) + M,$$

gdzie zapis $A \oplus M$ oznacza wykonanie XOR każdego bitu A_i z tym samym sygnałem M. Dla $M = 0$ otrzymujemy $Y = A$, natomiast dla $M = 1$:

$$Y = \overline{A} + 1,$$

czyli uzupełnienie do dwóch (modulo 16). Poniższy schemat przedstawia realizację układu 4-bitowego dla wejść A_0–A_3 i wyjść Y_0–Y_3.



8.4 Przykładowa weryfikacja

M	A	Y
0	0101	0101
1	0101	1011
1	0000	0000
1	1111	0001
0	1010	1010

8.5 Wnioski

Sterowanie XOR i przeniesieniem wejściowym pozwala uzyskać $Y = A$ dla $M = 0$ lub uzupełnienie do dwóch dla $M = 1$.

9 Zadanie 8 – Detekcja liczby z przedziału 5–10

9.1 Treść zadania

Zaprojektować układ, który sprawdza, czy 4-bitowa liczba wejściowa należy do przedziału 5–10. Można wykorzystać komparator scalony 7485.

9.2 Tabela prawdy

Przyjęto standardowo X_3 jako bit najbardziej znaczący, a X_0 jako najmniej znaczący.

wartość	X_3	X_2	X_1	X_0	Y
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	0
3	0	0	1	1	0
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	1
10	1	0	1	0	1
11	1	0	1	1	0
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	0

9.3 Minimalizacja

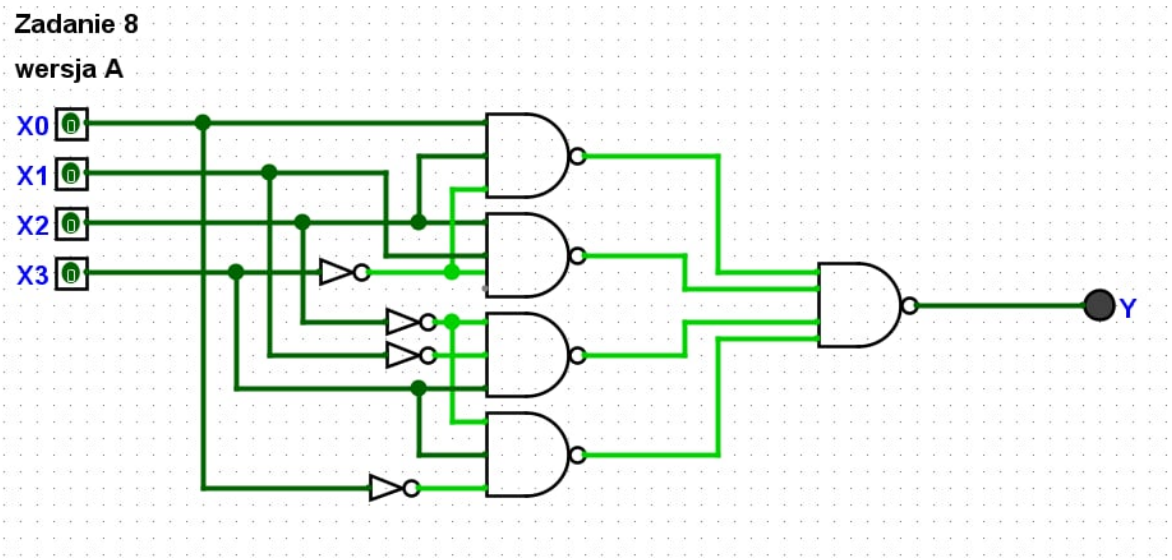
Dla kolejności zmiennych X_3, X_2, X_1, X_0:

$$Y = \Sigma m(5, 6, 7, 8, 9, 10).$$

$$\begin{aligned} Y &= \overline{X_3}X_2(\overline{X_1}X_0 + X_1\overline{X_0} + X_1X_0) \\ &\quad + X_3\overline{X_2}(\overline{X_1X_0} + \overline{X_1}X_0 + X_1\overline{X_0}) \\ &= \overline{X_3}X_2(X_0 + X_1\overline{X_0}) + X_3\overline{X_2}(\overline{X_1} + X_1\overline{X_0}) \\ &= \overline{X_3}X_2(X_1 + X_0) + X_3\overline{X_2}(\overline{X_1} + \overline{X_0}) \\ &= \overline{X_3}X_2X_0 + \overline{X_3}X_2X_1 \\ &\quad + X_3\overline{X_2X_0} + X_3\overline{X_2X_1}. \end{aligned}$$

9.4 Wersja A – realizacja z bramek

Cztery trójwejściowe bramki NAND tworzą zanegowane składniki iloczynowe, a końcowa bramka NAND wykonuje ich sumę zgodnie z prawem de Morgana.



9.5 Wersja B – realizacja z komparatorami 7485

Dwa komparatory 7485 sprawdzają granice przedziału:

$$Y = (X \geq 5) \cdot (X \leq 10).$$

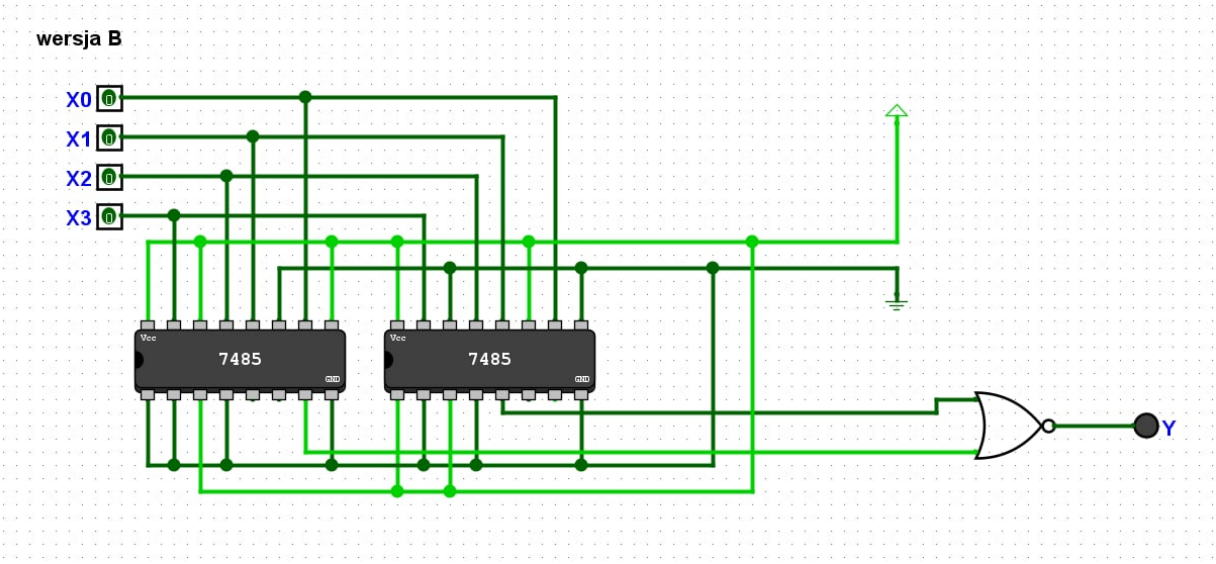
Równoważnie można wykrywać stany spoza zakresu i zanegować ich sumę:

$$Y = \overline{(X < 5) + (X > 10)}.$$

Wejścia kaskadowe obu komparatorów ustawia się jako:

$$I_{A>B} = 0, \qquad I_{A<B} = 0, \qquad I_{A=B} = 1.$$

Bramka NOR łączy wyjścia $X < 5$ i $X > 10$, dając $Y = 1$ wyłącznie wewnątrz przedziału.



9.6 Weryfikacja

X	Y
0100 (4)	0
0101 (5)	1
0111 (7)	1
1000 (8)	1
1010 (10)	1
1011 (11)	0
1111 (15)	0

9.7 Wnioski

Wersja A (NAND–NAND) i wersja B (dwa komparatory 7485) wykrywają wartości od 5 do 10 włącznie.

10 Wnioski końcowe

Tabele prawdy i zależności logiczne posłużyły do wyznaczenia funkcji realizowanych bramkami. Kaskadowe łączenie sumatorów umożliwiło dodawanie liczb 4-bitowych, a negacja bitów i dodanie jedynki – wyznaczanie uzupełnienia do dwóch. Zastosowanie układu 7485 upraszcza realizację operacji arytmetycznych oraz porównywania liczb.